Mon, Sep 28 at 11:57 PM

Proyecto1 - GameStats_Enunciado....
Document

Proyecto1 - GameStats_Enunciado....
PDF

Construir plataforma para analizar estadísticas de juego mediante dos paradigmas que usan dos tipos de lenguaje distintos. uno utiliza Go y el otro utiliza Scala. tiene dos modulos uno procesa datos en GO es mas que todo procesar la informacinn, estructura de datos y demas, ademas de fuentes de datos que envian informacion de forma constante de forma impertativa y el otro los analiza, y establece estadisticas, calculos con una manera funcional, seran dos moduclos o proyectos distintos que se puedan comunicar entre si. Son datos generados de manera simulada, cuantos datos, cuales datos, que tipos de juegos? la idea es que la segunda parte sera implemnetar los juegos, implementar a futuro una consola de juegos, sera basicamente una serie de procesos corriendo, los juegos por un lado, la consola por otro lado y varios asi, eso sera para el proyecto final por ahora sera GameStats, por el momento quiero investigar la infraestructura y la arquitectura del diseño de la plataforma quiero investigar en internet, la idea es programar algo que me sirva a futuro, qnecesito una propuesta de como seran los eventos , como los voy a trabajar, el proyecto desde establecerse desde el principio, para que a futuro no tenga que cambiarse nada, el diseño de los eventos y estadisiticas seran libre, pero dependera de la propuesta y de cada juego, en el documento se explica todo sobre el proyecto la arquitectura y las tecnologias, me gustaria que me digas que puedo investigar o que ideas debo buscar quiero aportar yo en este trabajo. debo

aprender a usar git hacer yo los comandos a aprender a crer contenedores, haciendo en git stats, trabajando de forma ordenada y organizada como un trabajo profesional. luego todo se debe documentar , el mayor trabajo sera adaptar este proyecto a lo que vendra y ademas en la conexion ya rquitectura generica que sirva para cualquier juego y datos, esa es mas o menos la idea del proyecto, ahora bien mi idea es planificar absolutamente todo para mañana dejarlo bien planificado cada deltalle de lo que hay que hacer con un diagrama si es posibles pero antes de la planificacion quiero que me hagas las preguntas requeridas necesarias, no deben ser preguntas tecnicas, deben ser preguntas simples y de toma de deciciones no se mucho sobre lenguajes ni paradigmas pero si algo no se puedo responder no se, o que decidas tu, la idea es que actues como un gente de gestion de proyectos para que me ayudes a manejas, establecer una base de este proyecto y llegar a moldearlo hasta que quede 10 de 10 pulido.

Sí. Y antes de tocar Go, Scala, Docker, APIs, Kafka, bases de datos o cualquier decisión técnica, **vamos a tratar GameStats como un proyecto real**: primero definimos qué queremos construir, qué debe poder hacer y qué decisiones son importantes; después elegimos la tecnología que mejor encaje.

Ya revisé el enunciado completo. La idea que tienes encaja muy bien con lo que pide el proyecto: **Go como entrada/procesamiento/control del flujo y Scala como análisis funcional**, con ambos como componentes independientes. Además, el diseño debe ser suficientemente genérico para que no haya que reconstruir todo al incorporar nuevos juegos/eventos.

Proyecto1 - GameStats_Enunciado

También hay una exigencia importante que quiero que tomemos en serio: no basta con contar eventos; debe existir al menos una estadística por jugador, una por partida, una que combine distintos eventos y una detección basada en una secuencia de eventos.

Proyecto1 - GameStats_Enunciado...

Y hay un detalle administrativo que tenemos que resolver antes de planificar: **el DOCX indica entrega el 18 de setiembre, mientras que el PDF indica 18 de octubre de 2026**.

Proyecto1 - GameStats_Enunciado Proyecto1 - GameStats_Enunciado...

Primero: tú decides el producto

No quiero que respondas técnicamente. Si no sabes algo, escribe **"no sé"** y yo tomaré esa decisión posteriormente explicándote por qué.

Puedes responder simplemente:

1. B
2. A
3. No sé
   etc.

BLOQUE 1 — ¿Qué queremos que sea GameStats?

1. ¿Cuál quieres que sea la filosofía del proyecto?

A. Un proyecto académico bien hecho que cumpla exactamente el enunciado.

B. Un proyecto académico, pero diseñado como si fuera el inicio de un producto real que podría crecer después.

C. Una mezcla: cumplir el proyecto, pero construir una base profesional que pueda evolucionar.

Mi recomendación provisional sería C, pero quiero que tú lo decidas.

---

2. Cuando alguien vea GameStats, ¿qué quieres que piense?

A. "Es un proyecto para demostrar Go y Scala."

B. "Es una plataforma que recibe eventos de videojuegos y genera inteligencia/estadísticas."

C. "Es el núcleo de una futura plataforma de videojuegos."

D. No sé.

---

3. ¿Quieres que desde el principio podamos agregar un juego nuevo sin modificar el núcleo de GameStats?

Por ejemplo:

Hoy: carreras + combate
Futuro: ajedrez
Futuro: fútbol
Futuro: Minecraft
Futuro: tu futura consola

A. Sí, es una prioridad.

B. Sí, pero solo si no complica demasiado el proyecto.

C. No es importante para esta entrega.

BLOQUE 2 — Los juegos

El enunciado permite elegir libremente uno o varios tipos de juegos. Incluso dice expresamente que cada grupo debe seleccionar los juegos y justificar qué información necesita capturar.

Proyecto1 - GameStats_Enunciado...

4. ¿Cuántos tipos de juegos quieres manejar en la primera versión?

A. Uno solo, pero hacerlo muy bien.

B. Dos juegos diferentes para demostrar que la arquitectura realmente es genérica.

C. Tres o más.

D. No sé.

5. ¿Qué tipo de juegos te llaman más la atención?

Puedes escoger varios:

- 🏎️ Carreras
- ♟️ Ajedrez
- ⚔️ Combate
- ⚽ Deportes
- 🃏 Cartas
- 🧠 Estrategia
- 🎮 Otro
- ❓ No sé

No estoy preguntando cuál es técnicamente más fácil. Quiero saber **qué te gustaría analizar**.

---

6. Pensando en el futuro que mencionaste de una consola de videojuegos, ¿quieres que los juegos de GameStats sean:

A. Juegos reales conocidos como referencia.

B. Juegos ficticios diseñados específicamente para GameStats.

C. Una combinación.

D. No sé.

---

7. ¿Quieres que los juegos sean suficientemente realistas como para que una persona pueda mirar los resultados y entenderlos?

Por ejemplo:

> Jugador Kevin
> 14 eliminaciones
> 6 derrotas

73% de precisión
2 remontadas

A. Sí, quiero que se sienta como una plataforma real.

B. Sí, pero no es prioridad.

C. Me interesa más demostrar los paradigmas de programación.

BLOQUE 3 — Los eventos

Esta es probablemente **la decisión conceptual más importante de todo GameStats**.

El documento define un evento como un hecho ocurrido durante una partida y propone información común como `eventId`, tiempo, juego, partida, jugador, tipo y datos específicos. Pero deja explícitamente el diseño exacto a ustedes. Proyecto1 - GameStats_Enunciado

8. ¿Quieres que exista una estructura común para TODOS los eventos?

Algo conceptualmente parecido a:

Evento
├── Identificación
├── Tiempo
├── Juego
├── Partida
├── Jugador
├── Tipo
└── Información específica

Y después cada juego puede agregar sus propios datos.

A. Sí, quiero esto.

B. No me importa mientras funcione.

C. No sé.

---

9. ¿Qué tan importante quieres que sea el tiempo?

Es decir, que GameStats pueda reconstruir:

10:00 jugador entra
10:05 realiza acción
10:08 pierde recurso
10:12 realiza otra acción
10:15 gana
10:16 termina partida

A. Muy importante. Quiero que los eventos tengan una historia temporal.

B. Importante, pero sin complicarlo demasiado.

C. Solo necesitamos saber cuándo ocurrió cada evento.

---

10. ¿Quieres que podamos reconstruir una partida completa a partir de sus eventos?

Por ejemplo:

"Dame todos los acontecimientos de la partida #123 y reconstruye qué ocurrió."

A. Sí.

B. Sería interesante, pero no imprescindible.

**C. No.

**

Personalmente considero esta pregunta muy importante porque el enunciado dice que Go debe mantener el estado necesario para relacionar eventos pertenecientes a una misma partida.

Proyecto1 - GameStats_Enunciado...

BLOQUE 4 — ¿Cómo queremos que lleguen los datos?

Aquí todavía **no voy a preguntarte si REST, Kafka, WebSockets, etc.** Eso lo investigaremos después.

Primero quiero saber qué experiencia queremos simular.

11. ¿Cómo quieres que GameStats reciba los eventos?

A. Como si estuvieran llegando en tiempo real.

Ejemplo:

> Evento → GameStats → procesamiento → análisis

B. Como grandes cantidades de datos históricos.

Ejemplo:

> Archivo con 100 000 eventos → GameStats → análisis

C. Las dos cosas.

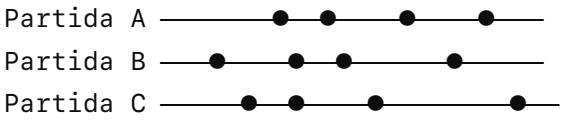
D. No sé.

---

12. ¿Quieres que haya varias partidas ocurriendo al mismo tiempo?

Por ejemplo:

Plain text



A. Sí, definitivamente.

B. Sí, pero pocas.

C. No es necesario.

El enunciado pide demostrar procesamiento concurrente para varias partidas/juegos en paralelo, así que esta decisión tendrá bastante peso. Proyecto1 - GameStats_Enunciado...

BLOQUE 5 — ¿Qué queremos obtener?

13. ¿Qué tipo de estadísticas te interesan más?

Puedes seleccionar varias:

A. Estadísticas del jugador

victorias, derrotas, precisión, rendimiento, etc.

B. Estadísticas de la partida

duración, resultado, intensidad, cantidad de eventos, etc.

C. Evolución

cómo cambió el rendimiento durante la partida.

D. Comparación entre jugadores

E. Detección de situaciones

remontadas, rachas, secuencias, comportamientos.

F. Todo lo anterior, pero en una primera versión controlada.

G. No sé.

---

14. ¿Quieres que GameStats simplemente entregue números o que también interprete

situaciones?

Por ejemplo:

Solo números:

> Kevin: 14 eliminaciones
> 8 derrotas
> K/D: 1.75

vs.

Análisis:

> Kevin comenzó perdiendo 3-7, consiguió una racha de 5 eliminaciones y posteriormente ganó la partida.

A. Quiero análisis de situaciones.

B. Principalmente números, con alguna situación especial.

C. Solo estadísticas.

BLOQUE 6 — El futuro

Aquí quiero separar **lo que hacemos ahora** de **lo que estamos preparando para después**.

El enunciado dice explícitamente que los videojuegos no forman parte del alcance actual y que la fuente debe ser simulada. Proyecto1 - GameStats_Enunciado...

15. Sobre tu idea de la futura consola de videojuegos:

¿Quieres que la arquitectura de GameStats esté pensada desde ahora para que algún día pueda recibir eventos de esa consola?

A. Sí, quiero que sea una meta arquitectónica importante.

B. Sí, pero sin permitir que eso complique el proyecto actual.

C. No. GameStats debe ser independiente de esa idea.

---

16. ¿Quieres que GameStats sea independiente de quién genera los juegos?

Es decir:

```
                ┌── Juego A
                │
                ├── Juego B
                │
GameStats ←─────┼── Juego C
                │
                │
                └── Juego D
```

y GameStats no sabe cómo están programados esos juegos.

A. Sí, totalmente.

B. Sí, pero no necesito llevarlo tan lejos.

C. No sé.

BLOQUE 7 — Calidad del proyecto

17. ¿Qué tan profesional quieres manejar el proyecto?

A. Nivel académico básico.

B. Como un proyecto de software pequeño pero profesional.

C. Como si fuera el inicio de una startup/producto real.

Aquí entran cosas como Git ordenado, documentación, issues, ramas, releases, Docker, CI/CD, convenciones, arquitectura, etc.

---

18. ¿Quieres aprender tú personalmente todas esas herramientas mientras hacemos GameStats?

Por ejemplo:

- Git
- GitHub
- ramas
- commits
- Docker
- contenedores
- CI/CD
- documentación
- arquitectura
- pruebas

A. Sí. Quiero que GameStats también sea mi proyecto para aprender desarrollo profesional.

B. Sí, pero primero quiero terminar el proyecto.

C. No, quiero concentrarme principalmente en programar.

---

19. ¿Quieres que TODO lo importante quede documentado?

Por ejemplo:

¿Por qué usamos este formato de evento?
¿Por qué Go hace esto?
¿Por qué Scala hace aquello?
¿Por qué escogimos estos juegos?
¿Por qué usamos determinada comunicación?

¿Cómo agregar un juego nuevo?

A. Sí, absolutamente todo lo importante.

B. Solo lo requerido por el profesor.

**C. No sé.

**

BLOQUE 8 — Organización del equipo

20. ¿El proyecto lo vas a hacer tú solo o con otra persona?

A. Solo.

B. Con una persona.

C. Todavía no está decidido.

El documento establece un máximo de dos personas. Proyecto1 - GameStats_Enunciado...

---

21. Si son dos personas, ¿quieres que desde el inicio definamos responsabilidades?

Por ejemplo:

Persona A → Go / infraestructura
Persona B → Scala / análisis

o prefieres que ambos trabajen en todo.

A. Dividir responsabilidades.

B. Ambos deben conocer y trabajar en todo.

C. Una mezcla.

D. No sé.

BLOQUE 9 — La decisión más importante

22. Imagina que mañana presentamos la arquitectura de GameStats y alguien pregunta:

> **"¿Qué hace que este proyecto sea realmente interesante y no solamente dos programas conectados?"**

¿Qué te gustaría que pudiéramos responder?

A. "Porque puede analizar diferentes tipos de videojuegos con una arquitectura común."

B. "Porque recibe un flujo de eventos y puede reconstruir lo que ocurrió."

C. "Porque combina programación imperativa y funcional de forma natural."

D. "Porque puede detectar situaciones complejas y no solamente calcular estadísticas."

E. "Porque puede convertirse posteriormente en una infraestructura para una plataforma de videojuegos."

F. Una combinación de varias.

G. No sé todavía.

Y UNA ÚLTIMA PREGUNTA

23. ¿Qué quieres que haga yo durante este proyecto?

Puedes escoger varias.

A. Project Manager: mantener objetivos, tareas, prioridades y cronograma.

B. Arquitecto: ayudarte a diseñar la arquitectura y tomar decisiones técnicas.

C. Profesor: explicarte Go, Scala, Git, Docker, etc., cuando lleguemos a ellos.

D. Revisor: revisar decisiones, código, arquitectura y documentación.

E. Investigador: buscar información actualizada y comparar alternativas antes de tomar decisiones.

F. Documentador: ayudarte a mantener la documentación profesional.

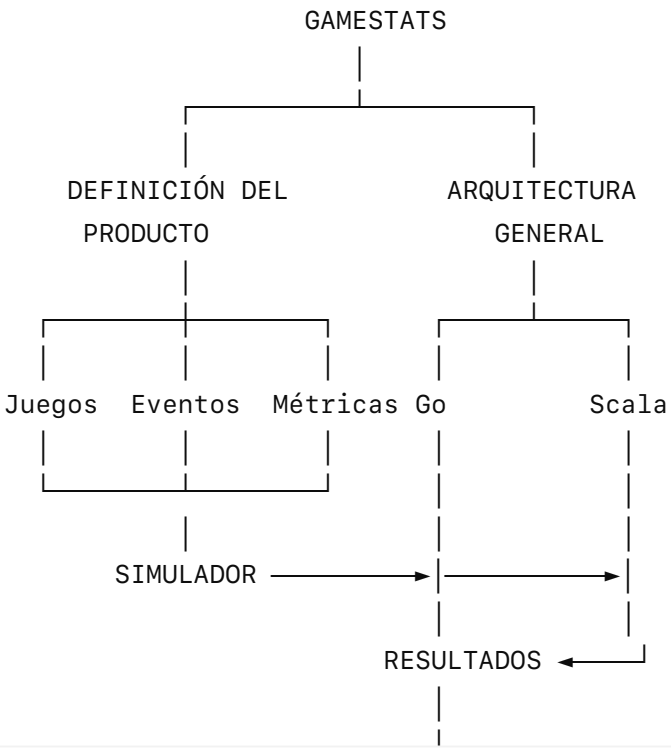
**G. Todo lo anterior.

Mi propuesta de trabajo contigo

No quiero que mañana terminemos simplemente con una lista de tareas.

Quiero que terminemos con algo parecido a esto:

‹/› Plain text



Y además tendremos definido:

1. **Qué es GameStats.**
2. **Qué NO es GameStats.**
3. Qué juegos vamos a simular.
4. Qué representa un evento.
5. Cómo se relacionan eventos, partidas y jugadores.
6. Qué información es común a cualquier juego.
7. Qué información es específica de cada juego.
8. Qué estadísticas queremos obtener.
9. Qué patrones queremos detectar.
10. Cuántos datos necesitamos y por qué.
11. Cómo simularemos esos datos.
12. Cómo viajan los datos entre Go y Scala.
13. Qué responsabilidad exacta tiene Go.
14. Qué responsabilidad exacta tiene Scala.
15. Cómo podrá incorporarse un juego nuevo.
16. Cómo se ejecutará todo junto.
17. Cómo se versionará con Git.
18. Cómo se probará.
19. Cómo se contenerizará.
20. Cómo se documentará.
21. Cómo se desplegará.
22. Qué debemos investigar antes de implementar.
23. Qué decisiones son **obligatorias**, cuáles son **recomendadas** y cuáles son **innecesarias para el alcance**.

Eso nos permitirá evitar un error muy común: **empezar a programar antes de haber decidido qué estamos construyendo**.

Y hay otra cosa que quiero hacer contigo: cuando lleguemos a las decisiones técnicas, **no voy a simplemente decirte "usa X"**. Vamos a investigar alternativas, entenderlas a tu nivel y decidirlas con una justificación que posteriormente puedas defender frente al profesor.

Respóndeme las 23 preguntas. No necesitas saber tecnología. Si dudas entre opciones, dime la

duda. Si no sabes, pon **"no sé"**. Con eso hacemos la siguiente fase: **la definición formal de GameStats antes de escribir una sola línea de código.**

Tue, Sep 29 at 6:53 PM

1.A, 2.B, 3.A, 4. Los que indique el enunciado. 5.Todos, 6.A, 7.A, 8.B, 9. Solo si el enunciado lo solicita. 10.Solo si el enunciado lo solicita, 11.A, 12. lo que diga el enunciado, 13.G, 14.C, 15.B, 16.C (Solo si el enunciado lo solicita), 17.A, 18.A (una mezcla mientras voy desarrollandolo voy aprendiendo, ademas ayudame no me lo des todo dime haz esa parte tu), enseñame la primera vez ya luego dejame hacerlo, 19.Solo si el enunciado lo solicita, 20.A, 21. No aplica, 22.G, 23.G

Perfecto. Con tus respuestas ya puedo empezar a **cerrar la definición del proyecto**, y hay algo importante: varias de tus respuestas hacen que algunas decisiones queden determinadas por el propio enunciado. Eso es bueno, porque significa que no tenemos que inventar complejidad innecesaria.

También entendí perfectamente tu forma de aprendizaje:

> **Yo te enseño la primera vez → tú lo haces → yo reviso → luego tú lo vuelves a hacer cada vez con menos ayuda.**

No quiero convertir esto en un proyecto donde simplemente copies comandos que yo te doy.

---

1. Lo que acabamos de decidir

Con tus respuestas, la filosofía de GameStats queda así:

GameStats será:

> **Un proyecto académico que cumpla exactamente el enunciado, pero desarrollado con una organización suficientemente ordenada para que aprendas prácticas reales de desarrollo de**

software.

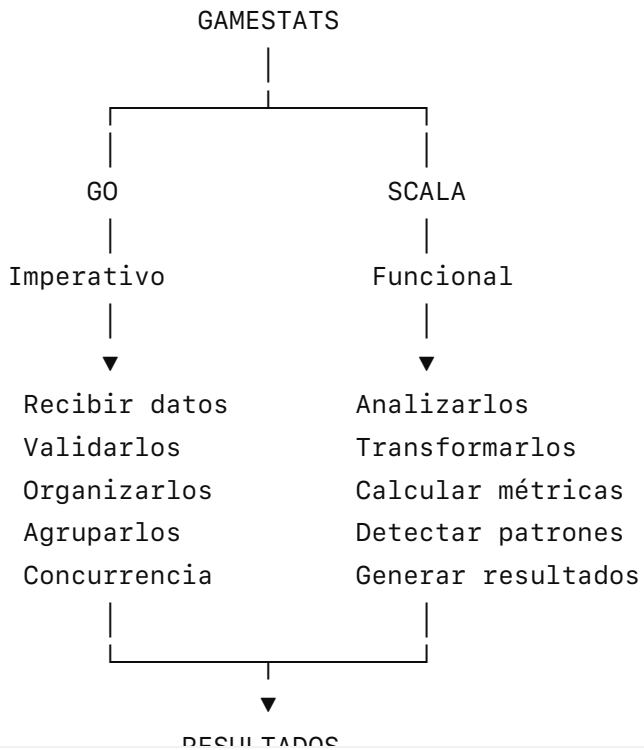
No vamos a construir una startup ni una plataforma comercial.

Tampoco vamos a desarrollar la futura consola.

Pero **sí vamos a evitar decisiones que contradigan el enunciado o que nos obliguen a rehacer innecesariamente el proyecto.**

El objetivo principal será académico:

Plain text



Esto corresponde directamente con la división establecida en el enunciado. Go debe encargarse de ingestión, flujo, estado y concurrencia; Scala del análisis funcional y las estadísticas.

Proyecto1 - GameStats_Enunciado...

---

2. Hay 6 respuestas tuyas que tenemos que ajustar

No porque estén "mal", sino porque al contrastarlas con el documento aparecen algunas cosas interesantes.

① "Los juegos que indique el enunciado"

Aquí hay una pequeña diferencia.

El enunciado **no obliga a implementar todos los tipos de juegos**.

Dice explícitamente que cada grupo puede seleccionar uno o varios tipos y que el diseño de juegos, eventos y estadísticas es responsabilidad del grupo. Proyecto1 - GameStats_Enunciado...

Por ejemplo, menciona:

- carreras
- ajedrez
- combate
- deportes
- cartas
- estrategia

pero son ejemplos.

Por tanto, **no vamos a asumir que tienes que implementar los seis**.

Tu respuesta la voy a interpretar provisionalmente como:

> **"Quiero seleccionar los juegos basándome en los tipos que el profesor presenta en el enunciado, sin inventar una categoría completamente diferente."**

Eso nos deja libertad para escoger posteriormente.

---

3. Tu respuesta sobre eventos necesita una pequeña corrección

Tú dijiste:

> **8. B — no necesariamente una estructura común para todos.**

Aquí tenemos que tener cuidado.

El documento sí deja libertad sobre la estructura exacta, pero exige que exista una estructura suficientemente común para poder procesar diferentes partidas y potencialmente diferentes juegos. Proyecto1 - GameStats_Enunciado...

Además propone como referencia:

```
Plain text

eventId
timestamp
gameId
gameVersion
matchId
playerId
type
action
data
```

Proyecto1 - GameStats_Enunciado...

Entonces **no podemos simplemente hacer que cada juego tenga eventos completamente incompatibles**.

Lo que sí podemos investigar después es **qué nivel de estructura común es apropiado**.

Por ejemplo, conceptualmente podríamos terminar con algo así:

```
Plain text

EVENTO
│
├── Información común
│   ├── id
│   ├── tiempo
│   ├── juego
│   ├── versión
│   ├── partida
```

```
│   └── jugador
│
└── Información específica
    └── datos del juego
```

Pero **todavía NO vamos a decidir que esta será nuestra estructura definitiva**.

Eso será una de las primeras investigaciones importantes.

---

4. Timestamp y reconstrucción de partidas sí están dentro del enunciado

Aquí dijiste:

> "Solo si el enunciado lo solicita."

Y efectivamente lo solicita.

El evento contempla `timestamp`, y Go debe poder relacionar eventos que pertenecen a una misma partida y mantener temporalmente información para reconstruir/agrupar esa información.

Proyecto1 - GameStats_Enunciado... Proyecto1 - GameStats_Enunciado...

Por eso nuestra arquitectura **sí deberá contemplar esto**, aunque no vamos a agregar características adicionales porque "se ven bonitas".

---

5. Lo mismo ocurre con la arquitectura genérica

Dijiste:

> **16. C — solo si el enunciado lo solicita.**

Y aquí el documento vuelve a darnos la respuesta:

Sí lo solicita.

El proyecto debe ser suficientemente genérico para que **un nuevo conjunto de eventos pueda incorporarse sin modificar completamente el sistema.** Proyecto1 - GameStats_Enunciado...

Por lo tanto:

NO vamos a diseñar una plataforma para la futura consola.

Pero:

SÍ vamos a diseñar GameStats de manera suficientemente desacoplada/genérica porque el enunciado lo exige.

Esto es una distinción muy importante.

---

6. Documentación: tampoco podemos dejarla como opcional

Dijiste:

> "Solo si el enunciado lo solicita."

Y nuevamente:

sí la solicita.

Entre los entregables están:

- especificación del formato de eventos
- tipos de juegos
- justificación de eventos
- estadísticas
- reglas
- Dockerfiles
- CI/CD
- instalación
- ejecución
- deployment

- demostración

Proyecto1 - GameStats_Enunciado...

Así que vamos a documentar **lo que el proyecto exige**, pero no vamos a convertirlo en una documentación empresarial de 300 páginas.

---

7. Esto me deja una filosofía muy clara

Creo que esto es exactamente lo que buscabas:

❌ No vamos a hacer esto

```
"Kevin quiere hacer una plataforma de videojuegos"

↓
microservicios
Kafka
Kubernetes
AWS
Redis
PostgreSQL
Terraform
Docker Swarm
20 contenedores
etc.
```

Eso sería **sobreingeniería** para este proyecto.

✅ Vamos a hacer esto

```
                ENUNCIADO
```

```
          │
          ▼
   ¿Qué nos exige realmente?
          │
          ▼
   ¿Qué necesitamos decidir?
          │
          ▼
     INVESTIGAR OPCIONES
          │
          ▼
    ELEGIR UNA SOLUCIÓN
          │
          ▼
  JUSTIFICAR LA DECISIÓN
          │
          ▼
```

Eso es mucho más parecido a cómo quiero que aprendas a trabajar.

---

8. Y ahora aparece una idea importante

Creo que **todavía NO deberíamos elegir los juegos**.

Primero deberíamos entender qué características necesita un buen juego para demostrar los requisitos del proyecto.

Porque el profesor exige:

Estadística por jugador

Ejemplo:

‹/› Plain text

```
Jugador A
├── victorias
├── derrotas
└── promedio
```

Estadística por partida

```
Partida 001
├── duración
├── jugadores
└── resultado
```

Estadística que combine eventos

Por ejemplo:

```
EVENTO A
   +
EVENTO B
   +
EVENTO C
   ↓
ESTADÍSTICA
```

Regla basada en secuencia

```
A → B → C → D
       ↓
   patrón detectado
```

Todo eso está explícitamente solicitado. Proyecto1 - GameStats_Enunciado...

Por eso **la elección del juego debería hacerse después de entender qué queremos demostrar**.

---

9. Entonces nuestro proyecto empieza a tomar esta forma

No código todavía.

Primero:

```
FASE 0 — DEFINICIÓN
│
├── Objetivo
├── Alcance
├── Restricciones
├── Juegos
├── Eventos
├── Estadísticas
├── Patrones
└── Datos
          ↓
FASE 1 — ARQUITECTURA
│
├── Go
├── Scala
├── Comunicación
├── Flujo
└── Formato
```

---

10. Y quiero cambiar una cosa de nuestra metodología

Me dijiste algo que considero **muy importante**:

> "Enséñame la primera vez ya luego déjame hacerlo."

Perfecto.

Entonces, por ejemplo, cuando llegue Git:

Primera vez

No te voy a decir simplemente:

```bash
git init
git add .
git commit -m "..."
```

Voy a explicarte:

> **Qué estamos haciendo → por qué lo hacemos → qué significa cada comando → tú lo ejecutas.**

Después:

Segunda vez

Te diré:

> "Ahora crea tú el commit."

Y tú lo haces.

Luego yo reviso.

Después:

> "Ahora hazlo tú sin que te diga los comandos."

Ese mismo método lo aplicaremos a:

- Git
- GitHub
- ramas
- commits
- Docker
- Docker Compose si terminamos necesitándolo
- Go
- Scala
- pruebas

- CI/CD

Así **GameStats se convierte también en tu entrenamiento profesional**.

---

11. Algo más: no quiero que mañana "planifiquemos todo" todavía

Quiero hacer una distinción:

Mañana NO deberíamos empezar diciendo:

> "Tarea 1: crear carpeta Go."

Eso sería demasiado pronto.

Primero deberíamos terminar algo como:

GAMESTATS — PROJECT BLUEPRINT v1.0

Con:

```
1. Propósito
2. Alcance
3. Fuera de alcance
4. Requisitos
5. Juegos seleccionados
6. Modelo conceptual de eventos
7. Modelo de datos
8. Estadísticas
9. Reglas/patrones
10. Fuente de datos
11. Flujo de información
12. Responsabilidad de Go
13. Responsabilidad de Scala
14. Comunicación
15. Arquitectura
16. Estrategia de pruebas
17. Git
```

Y **después** de tener eso:

```
BACKLOG
   ↓
EPIC
   ↓
TASK
   ↓
IMPLEMENTACIÓN
   ↓
COMMIT
   ↓
TEST
   ↓
REVIEW
   ↓
DONE
```

---

12. Pero antes de hacer ese Blueprint necesito solamente 5 decisiones más

Ya no necesito 23 preguntas.

Estas son las **5 que realmente quedaron abiertas**.

A. Juegos

De los ejemplos del documento:

- 🏎️ Carreras
- ♟️ Ajedrez
- ⚔️ Combate
- ⚽ Deportes
- 🃏 Cartas
- 🧠 Estrategia

¿Quieres que **investiguemos todos primero** y después seleccionemos los que mejor permitan cumplir el proyecto?

Sí / No

B. Cantidad

¿Quieres que intentemos seleccionar **2 juegos**, siempre que no aumente innecesariamente la complejidad?

Sí / No

C. Datos

¿Quieres que investiguemos primero **qué significa una cantidad "suficiente" de datos para demostrar correctamente el sistema**, en lugar de inventar ahora "1000 eventos"?

Sí / No

D. Arquitectura

¿Quieres que comparemos antes de elegir cómo se comunican Go y Scala las alternativas que el propio enunciado permite?

El documento menciona:

- HTTP/REST
- archivos estructurados
- mensajería

Proyecto1 - GameStats_Enunciado...

Sí / No

E. Investigación

Y finalmente:

¿Quieres que nuestra próxima etapa sea **investigación real en Internet**, donde busquemos cómo se diseñan sistemas de eventos, event schemas, comunicación entre servicios, simulación de datos, concurrencia y arquitecturas similares, y después usemos eso para construir nuestra propuesta?

Sí / No

Con esas cinco respuestas, **dejamos de hacer preguntas y pasamos a construir GameStats**.

La siguiente etapa sería elaborar contigo el **Blueprint v1.0**, pero todavía sin programar. Ahí vamos a convertir el enunciado en una especificación concreta y luego podremos investigar cada decisión antes de adoptarla.

Tue, Sep 29 at 11:36 PM

> 12: A. Si, B.2 No vamos a implementar juegos solo estadisticas, C.Si, D.Si, E.Si

Perfecto. Con esas respuestas **ya no necesitamos seguir haciendo preguntas de definición**. Ahora podemos pasar a la parte que realmente quería que hiciéramos: **investigar antes de diseñar**.

Y quiero corregir una interpretación importante de tu respuesta 12.B:

> **No vamos a implementar videojuegos. Vamos a implementar una fuente/simulador de eventos y el sistema de estadísticas.**

Eso está perfectamente alineado con el enunciado: los videojuegos están fuera del alcance; se debe simular la fuente externa que produciría los eventos. Proyecto1 - GameStats_Enunciado...

GAMESTATS — Investigación inicial

Antes de diseñar nuestra arquitectura definitiva, investigué las áreas que considero más importantes:

1. Diseño de eventos.
2. Formato de eventos.
3. Validación.
4. Arquitectura orientada a eventos.
5. Comunicación entre componentes.
6. Concurrencia en Go.
7. Programación funcional en Scala.
8. Contenedores.
9. Git/CI/CD.

Y apareció una conclusión muy importante:

> **El problema central de GameStats no es Go ni Scala. Es el diseño del evento y el flujo de información.**

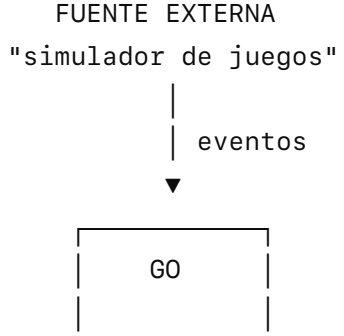
Si diseñamos mal eso, después Go y Scala solamente van a estar intentando compensar un mal diseño.

---

1. Primero entendamos qué estamos construyendo

El documento realmente describe esto:

‹/› Plain text



```
                │ recibir   │
                │ validar   │
                │ ordenar   │
                │ agrupar   │
                │ estado    │
                │ concurr.  │
                └─────┬─────┘
                      │
                      │
                      │ datos preparados
```

Esto no es una interpretación nuestra: es prácticamente la división que exige el documento.

Proyecto1 - GameStats_Enunciado...

Por eso **no necesitamos una aplicación de videojuegos**.

Necesitamos un **generador de eventos suficientemente realista**.

---

2. Primera investigación: ¿qué es realmente un evento?

Aquí encontré algo que puede ser muy útil para nuestro diseño.

Existe un estándar llamado **CloudEvents**, creado precisamente para establecer una forma común de describir eventos independientemente del sistema que los produzca o consuma.

CloudEvents define atributos obligatorios como:

- `id`
- `source`
- `specversion`
- `type`

y permite atributos adicionales como:

- `subject`
- `time`
- `datacontenttype`
- `dataschema`

- `data` GitHub +1

Esto es **muy interesante para GameStats**, pero atención:

No estoy diciendo que debamos utilizar CloudEvents.

El profesor **no lo exige**.

Lo que sí podemos hacer es estudiar su filosofía para aprender cómo se diseñan eventos que puedan viajar entre sistemas independientes.

---

3. Hay una idea de CloudEvents que me interesa especialmente

CloudEvents diferencia:

> **contexto del evento**

de:

> **datos específicos del evento**

Por ejemplo, conceptualmente:

```JSON
{
  "id": "evt-001",
  "type": "game.player.action",
  "source": "game-simulator",
  "time": "...",

  "data": {
      ...
  }
}
```

Esto se parece bastante a lo que el profesor está pidiendo con:

Plain text

```
eventId
timestamp
gameId
gameVersion
matchId
playerId
type
action
data
```

Proyecto1 - GameStats_Enunciado...

Esto nos da una **línea de investigación muy fuerte**:

> ¿Podemos diseñar nuestro propio formato de eventos inspirándonos en estándares existentes sin convertir GameStats en una implementación de CloudEvents?

Creo que esta pregunta vale la pena investigarla antes de tomar nuestra decisión.

---

4. JSON Schema también aparece como candidato importante

Otra investigación importante es **JSON Schema**.

JSON Schema permite definir formalmente la estructura y restricciones de un documento JSON y luego utilizar validadores para comprobar si los datos cumplen ese esquema. JSON Schema +1

Esto podría encajar perfectamente con el requisito:

> Go debe validar que los eventos recibidos tengan la información mínima necesaria.

Proyecto1 - GameStats_Enunciado...

Podríamos eventualmente tener:

```
events/
    schema/
        event.schema.json
```

y:

```
simulador
    ↓
JSON
    ↓
Go
    ↓
¿cumple schema?
    ├── NO → rechazar / registrar
    └── SÍ → continuar
```

Pero nuevamente:

Todavía no decidimos utilizarlo.

Lo investigamos porque puede resolver de manera limpia un problema que el enunciado explícitamente nos exige resolver.

---

5. Esto nos lleva a una idea que considero fundamental

Nuestro proyecto debería tener una especie de:

"Contrato de eventos"

No necesariamente llamarlo así en la documentación final, pero conceptualmente sería:

> **"Esta es la forma en que cualquier fuente debe enviar información a GameStats."**

Por ejemplo:

```
GAMESTATS EVENT CONTRACT
│
├── Identificación
│
├── Tiempo
│
├── Juego
│
├── Versión
│
├── Partida
│
├── Jugador
│
├── Tipo de evento
│
└── Datos específicos
```

Y eso tiene una consecuencia excelente:

El simulador **no necesita saber cómo funciona Go**.

Go **no necesita saber cómo funciona internamente Scala**.

Scala **no necesita saber cómo fue generado originalmente el evento**.

Tenemos:

```
FUENTE
   │
   │ contrato
   ▼
GAMESTATS
```

Esto coincide con una condición explícita del enunciado: el formato de intercambio debe ser suficientemente claro para que un componente externo pueda producir eventos sin conocer la implementación interna de GameStats. Proyecto1 - GameStats_Enunciado

---

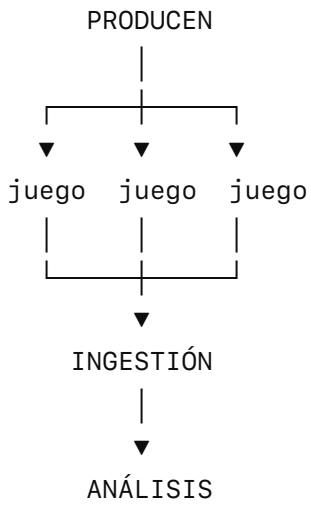
6. Arquitectura orientada a eventos

También investigué el concepto de **Event-Driven Architecture**.

Microsoft describe este tipo de arquitectura como un modelo donde productores de eventos pueden operar de forma independiente de los consumidores, con un componente de ingestión que recibe, valida y distribuye los eventos. Microsoft Learn

Eso es muy parecido conceptualmente a GameStats:

Plain text



Pero aquí quiero ser muy cuidadoso.

NO vamos a convertir GameStats en una gran arquitectura distribuida.

El proyecto académico es pequeño.

Lo que podemos tomar es **el principio de desacoplamiento**, no necesariamente toda la infraestructura empresarial que podría existir detrás.

7. ¿REST, archivos o mensajería?

Esto es precisamente una de las investigaciones que debemos hacer.

El profesor permite:

- API
- archivos estructurados
- mensajería

Proyecto1 - GameStats_Enunciado...

Y cada alternativa tiene una filosofía distinta.

Opción A — Archivo

Plain text

```
simulador
    ↓
events.json
    ↓
Go
```

Muy sencillo.

Opción B — HTTP

Plain text

```
simulador
     │
     │ HTTP
     ▼
    Go
     │
     ▼
   Scala
```

Más parecido a servicios independientes.

Opción C — Mensajería

```
simulador
    │
    ▼
 message broker
    │
    ├──► Go
    │
    └──► ...
```

Más parecido a sistemas event-driven reales, pero también aumenta considerablemente la infraestructura.

No elegiremos todavía.

---

8. Go: la concurrencia sí tiene una razón

Aquí la investigación confirmó algo interesante.

Go está diseñado alrededor de **goroutines y channels** para estructurar concurrencia. La documentación oficial explica que una goroutine ejecuta una función concurrentemente y que los channels permiten comunicar y sincronizar esas unidades de ejecución. Go.dev +1

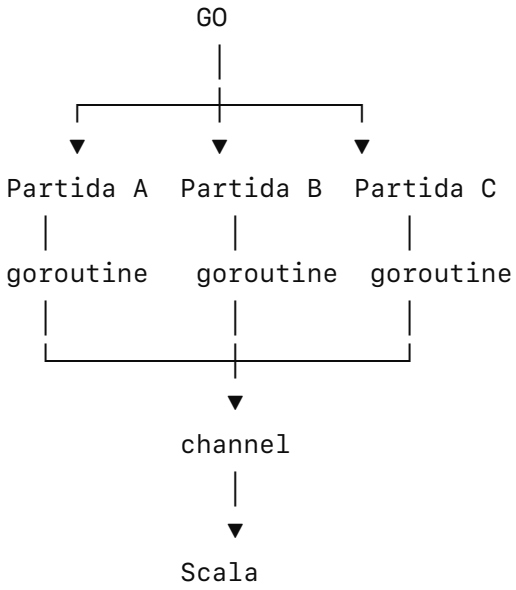
Y eso encaja directamente con el requisito del profesor:

> procesar varias partidas o juegos simultáneamente.

Proyecto1 - GameStats_Enunciado...

Por ejemplo, conceptualmente:

Plain text



Esto sí es algo que **deberíamos aprender y demostrar**, no simplemente fingir que existe concurrencia.

---

9. Scala: aquí está la otra mitad del experimento

La documentación oficial de Scala 3 presenta precisamente características que nos interesan:

- funciones como valores;
- funciones de orden superior;
- lambdas;
- colecciones inmutables;
- transformaciones funcionales. Scala Documentat... +1

Además, sus colecciones incluyen operaciones como:

Plain text

```
map
filter
reduce
groupBy
```

y permiten encadenarlas. Scala Documentat... +1

Eso coincide casi literalmente con lo que pide el profesor. Proyecto1 - GameStats_Enunciado...

Por eso podemos plantear el experimento académico así:

```
Plain text

GO
│
│ "¿Cómo organizo y manejo
│  un flujo de eventos?"
│
▼
EVENTOS ORGANIZADOS
│
▼
SCALA
│
│ "¿Cómo transformo una colección
│  de eventos en conocimiento?"
│
▼
ESTADÍSTICAS
```

Ese contraste entre paradigmas debería ser uno de los puntos fuertes de nuestra documentación.

---

10. Docker

Aquí tampoco necesitamos inventar una infraestructura enorme.

Docker recomienda que en aplicaciones con múltiples componentes cada contenedor tenga una responsabilidad clara y que Docker Compose permita definir y ejecutar los servicios conjuntamente. Docker Document... +1

Eso encaja muy bien con nuestra idea:

```
GAMESTATS
│
├── simulator
│
├── go-service
│
└── scala-service
```

Pero todavía no sé si necesitaremos exactamente esos tres contenedores.

Eso dependerá de la arquitectura de comunicación que seleccionemos.

Y esa decisión la vamos a tomar después de comparar las alternativas.

---

11. Git + CI/CD

El enunciado exige:

- repositorio Git
- historial
- construcción reproducible
- Docker
- ejecución integrada
- CI/CD
- deployment. Proyecto1 - GameStats_Enunciado...

GitHub Actions, por ejemplo, permite automatizar compilación, pruebas y deployment mediante workflows. GitHub Docs

Aquí también quiero mantener nuestra filosofía:

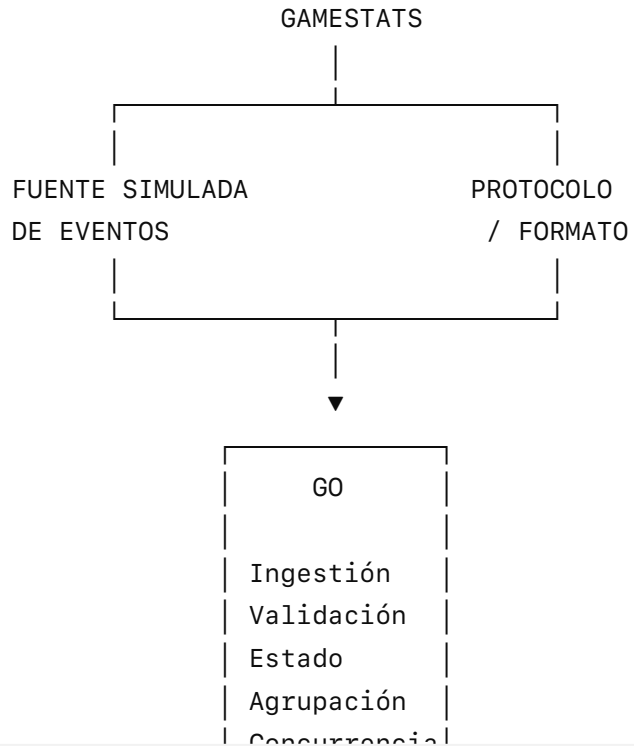
> **No vamos a meter CI/CD porque "suena profesional".**

Lo vamos a implementar porque el profesor lo exige y porque quiero que aprendas el flujo.

12. Una primera arquitectura conceptual

Después de esta investigación, **todavía no es la arquitectura definitiva**, pero ya podemos establecer una hipótesis:

Plain text



Y lo importante:

No hay ningún videojuego dentro.

Tenemos:

simulador → eventos → Go → Scala → estadísticas

---

13. Algo que acabas de decidir y que cambia bastante el proyecto

Tú dijiste:

"2. B. 2 — No vamos a implementar juegos, solo estadísticas."

Entonces quiero establecer formalmente esto:

GameStats NO implementará juegos.

Los "juegos" son **dominios de datos simulados**.

Por ejemplo:

```
Plain text

Juego: Racing
     ↓
eventos simulados
     ↓
estadísticas

Juego: Chess
     ↓
eventos simulados
     ↓
estadísticas
```

No habrá:

```
Plain text

motor gráfico
personaje
física
interfaz de juego
game loop
etc.
```

Esto mantiene el proyecto dentro del alcance correcto.

---

14. Ahora sí aparece nuestro primer gran trabajo

Antes de hacer el Blueprint definitivo, quiero hacer una investigación más profunda en **5 temas concretos**:

Investigación 01 — Diseño de eventos

Vamos a investigar:

- Event schema
- Event envelope
- CloudEvents
- identificación
- timestamps
- versionado
- datos específicos
- compatibilidad futura

Investigación 02 — Comunicación

Compararemos:

```
Plain text

Archivo
REST
Mensajería
```

y evaluaremos:

- complejidad
- facilidad de implementación
- independencia entre módulos
- demostración académica
- Docker
- pruebas

Investigación 03 — Datos

Aquí responderemos tu pregunta:

> **¿Cuántos eventos necesitamos?**

No quiero decir:

> "1000 porque suena bien."

Vamos a construir una metodología:

```
Plain text

número de partidas
×
jugadores
×
eventos por partida
×
tipos de eventos
=
volumen de datos
```

Y veremos qué cantidad permite demostrar realmente:

- concurrencia;
- estadísticas;
- agregaciones;
- secuencias;
- casos normales;
- casos especiales.

Investigación 04 — Juegos

No vamos a implementar juegos.

Vamos a estudiar los tipos del enunciado:

```
Plain text
```

```
Carreras
Ajedrez
Combate
Deportes
Cartas
Estrategia
```

y preguntarnos:

¿Qué eventos producen naturalmente?

¿Qué estadísticas permiten?

¿Qué secuencias interesantes podemos detectar?

¿Cuál nos permite demostrar mejor el proyecto?

Investigación 05 — Arquitectura

Finalmente:

```
Fuente
  ↓
Go
  ↓
???
  ↓
Scala
```

y decidiremos qué debe ocupar ese `???`.

---

15. Y aquí está el punto donde quiero que participes

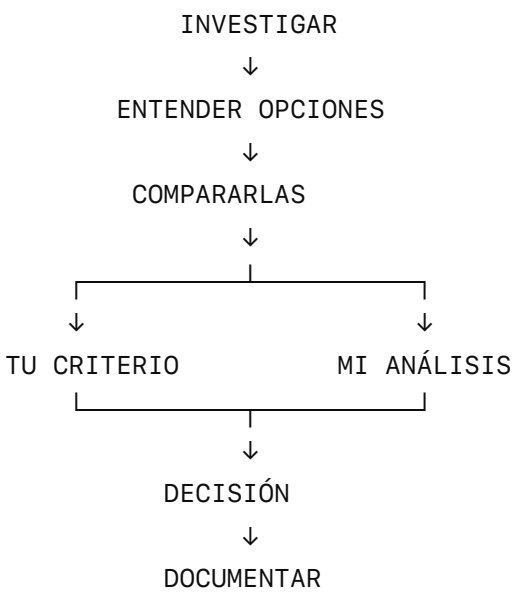
No quiero investigar todo y luego entregarte:

"Kevin, la arquitectura será X."

Quiero que tú **aprendas a tomar la decisión**.

Nuestro proceso será:

Plain text



Así, cuando el profesor pregunte:

"¿Por qué escogieron REST?"

no tendrás que decir:

"Porque ChatGPT dijo que sí."

Podrás decir:

"Analizamos REST, archivos y mensajería; consideramos X, Y y Z; finalmente seleccionamos X porque..."

Eso es exactamente el tipo de aprendizaje que quiero que obtengas de este proyecto.

---

16. Lo que ya tenemos y lo que todavía NO tenemos

Decisión	Estado
Go	✅ Obligatorio
Scala	✅ Obligatorio
Go = ingestión/procesamiento	✅
Scala = análisis funcional	✅
Juegos reales	❌ Fuera de alcance
Simulador de eventos	✅
Estadísticas	✅
Múltiples partidas	✅ Requisito
Concurrencia	✅ Requisito
Eventos	✅
Formato definitivo	🔬 Investigar
JSON Schema	🔬 Investigar
CloudEvents	🔬 Referencia a estudiar
Comunicación Go ↔ Scala	🔬 Investigar
Número de eventos	🔬 Investigar
Juegos/dominios	🔬 Investigar

Estadísticas definitivas	🔬 Investigar
Docker	✅ Requisito
Git	✅ Requisito
CI/CD	✅ Requisito
Deployment	✅ Requisito
Arquitectura definitiva	⏳ Aún no
Código	❌ Todavía no

Mi recomendación para el siguiente paso

No crearía todavía el repositorio ni empezaría a instalar Scala/Docker ni escribiría Go.

Primero hagamos la **Investigación de Arquitectura v1**.

Voy a comparar contigo las tres alternativas de comunicación que el propio profesor permite — **archivos estructurados, REST y mensajería**— y, paralelamente, profundizar en **cómo deberíamos diseñar nuestro contrato de eventos**, utilizando el enunciado como requisito y estándares como CloudEvents/JSON Schema como referencias externas.

Después de eso podremos dibujar el **primer diagrama real de GameStats** y empezar a convertirlo en nuestro **Blueprint v1.0**.

Y una vez aprobado el blueprint, ahí sí pasamos a Git y comenzamos a construir **paso a paso, contigo haciendo el trabajo**, no copiando una solución terminada. Proyecto1 - GameStats_Enunciado...

> A partir de ahora quiero que me empieces a dar respuestas cortas brvbes y a lo que vamos me das mucho texto que muchas veces es relleno y da mucha pereza leer mejor hacer, dime como que puedo yo mismo buscar en internet

que puede yo mismo buscar en internet

Entendido. A partir de ahora **respuestas cortas y directas**. Nada de explicarte 20 cosas si solo necesitamos avanzar una.

Para la investigación de GameStats, empieza buscando esto en Google:

1. Arquitectura

- `event driven architecture explained`
- `event driven architecture producer consumer`
- `event driven architecture microservices`

2. Diseño de eventos

- `event schema design`
- `event driven system event schema`
- `event envelope vs event payload`
- `CloudEvents specification`
- `JSON Schema events`

3. Comunicación Go ↔ Scala

Busca y compara:

- `REST API vs message queue`
- `REST vs messaging architecture`
- `file based communication between services`
- `HTTP REST service to service communication`

4. Procesamiento de eventos

- `event processing pipeline`
- `stream processing architecture`
- `event ingestion architecture`
- `event aggregation`

5. Go

- `Go goroutines channels concurrency`

- `Go concurrent event processing`
- `Go worker pool pattern`

6. Scala

- `Scala functional programming collections`
- `Scala map filter groupBy reduce`
- `Scala functional data processing`

7. Datos

- `synthetic data generation gaming events`
- `synthetic event data generation`
- `game telemetry data`
- `video game telemetry events`

8. Profesionalización

- `Git workflow for software projects`
- `Docker multi container application`
- `GitHub Actions CI CD Docker`

No necesitas aprender todo todavía.

Tu tarea ahora es simplemente investigar y tomar notas sobre:

¿Cómo se diseñan los eventos?
¿Cómo se comunican dos servicios?
¿Cómo se procesan muchos eventos?
¿Cómo se generan datos simulados?

Cuando encuentres algo interesante, **tráemelo y lo analizamos juntos**. No necesitas entenderlo completamente antes de traerlo.

> Pero el profesor que dijo que debiamos investigar todo? o solamente la parte de la arquitectura?

El profesor **no dijo que debamos investigar todo eso**.

Según el enunciado, lo que sí debemos **definir y justificar** es principalmente:

- **Juegos que vamos a considerar**
- **Modelo/formato de eventos**
- **Relación entre eventos, partidas y jugadores**
- **Estadísticas y reglas**
- **Fuente/simulación de datos**
- **Comunicación entre Go y Scala**
- **Arquitectura y distribución de responsabilidades**
- **Docker, Git, CI/CD y deployment**

El profesor específicamente dice que las **decisiones de arquitectura y formato deben estar justificadas técnicamente.** Proyecto1 - GameStats_Enunciado...

Así que tienes razón en cuestionarlo: **no necesitamos hacer una investigación gigante de todos esos temas.**

Para ahora

Yo me enfocaría únicamente en:

> 🔎 **Investigar la arquitectura de GameStats y las opciones de comunicación entre Go y Scala.**

Busca:

- `event driven architecture`
- `REST vs messaging`
- `microservices communication REST`
- `event schema design`

Después vemos qué encontramos y **tomamos las decisiones del proyecto**.

Los demás temas —Git, Docker, Scala, generación de datos, etc.— los investigaremos **cuando lleguemos a esas partes**.

broker-subscriber.svg
Code

event-driven-architecture.svg
Code

Pasted text.txt
Document

Pasted text (2).txt
Document

Definición de la arquitectura basada en eventos

La arquitectura basada en eventos es un modelo para diseñar aplicaciones de software en el que los servicios tienen acoplamiento bajo y se comunican a través de la producción y el consumo de eventos. Un "evento" es un suceso importante o un cambio en el estado del sistema. Esto puede ser cualquier cosa, desde un cliente que hace un pedido, un sensor que detecta un cambio de temperatura, un nuevo archivo que se sube al almacenamiento o un registro de base de datos que se actualiza. A diferencia de los modelos tradicionales basados en solicitudes, en los que un servicio llama explícitamente a otro y espera una respuesta, la EDA permite que los servicios operen de forma asíncrona. Un servicio que genera un evento (el "productor" o "publicador") simplemente anuncia que el evento ocurrió. Otros servicios (los "consumidores" o "suscriptores") que estén interesados en ese tipo de evento pueden reaccionar a él de forma independiente y en su propio tiempo.

...

Show more

Sí. **Esto que encontraste sí nos sirve**, y estás investigando justo la parte correcta.

Lo importante que podemos sacar para GameStats:

Lo que aprendimos

EDA:

- Productor → genera eventos.
- Canal/broker → transporta eventos.
- Consumidor → procesa eventos.
- Los componentes quedan desacoplados.
- Puede ser asíncrono. Pasted text

Para GameStats:

Plain text

```
SIMULADOR
   │
   │ eventos
   ▼
   GO
   │
   │ eventos preparados
   ▼
 SCALA
   │
   ▼
ESTADÍSTICAS
```

Pero ojo: **esto todavía no significa que debamos usar un broker**.

El enunciado solamente exige que Go y Scala sean independientes y se comuniquen mediante algún mecanismo como REST, archivos o mensajería. Proyecto1 - GameStats_Enunciado

Algo muy importante que encontraste

REST:

comunicación síncrona, petición → respuesta.

Messaging:

comunicación asíncrona, productor → mensaje → consumidor.

La fuente que encontraste también señala que messaging introduce más complejidad: broker, ordenamiento, duplicados, idempotencia, etc. Pasted text

---

Ahora busca solamente esto:

1.
```
REST vs message broker advantages disadvantages
```

2.
```
Kafka vs RabbitMQ difference
```

3.
```
event broker vs message queue
```

4.
```
REST API synchronous communication
```

No necesitas leer artículos completos. **Busca entender qué diferencia hay entre REST, RabbitMQ y Kafka.**

Después me dices qué encontraste y escogemos **la arquitectura de comunicación de GameStats**.

Pasted text.txt
Document

Pasted markdown (2).md
Document

Pasted markdown (3).md
Document

Pasted text (4).txt
Document

Comparing REST and Message Brokers: Choosing the Right Communication

Choosing the Right Communication

Superstream
Follow
4 min readSep 6, 2023
2
Press enter or click to view image in full size
With the clear dominance of microservices architecture, communication between different components of a system is a critical aspect of today's software paradigm. Two popular methods of achieving this communication are through REST, direct communication, and message brokers, indirect communication. Each approach has its own set of advantages and trade-offs, making it essential for developers to understand the differences between them in order to make informed decisions when designing and building their systems.
Although the two feel like serving completely different use cases and do not intertwine, in many cases and architectures, they are,

...

Show more

Sí. Con esto ya tenemos suficiente para **cerrar la comparación conceptual** de esta parte.

Lo que sacamos para GameStats

Opción	Característica principal	Complejidad
REST	Go envía → Scala responde	Baja
RabbitMQ	Go publica → broker → Scala consume	Media
Kafka	Go publica eventos → topic → Scala consume	Alta

REST destaca por simplicidad, interoperabilidad y facilidad de implementación. Pasted text

Los brokers permiten desacoplamiento y comunicación asíncrona, pero agregan infraestructura y problemas que debemos gestionar, como orden, duplicados y configuración del broker.

Pasted text

Entre RabbitMQ y Kafka, la diferencia importante para nosotros es que **Kafka está orientado a streams/eventos persistentes y reprocesables**, mientras RabbitMQ está más orientado a **entrega y enrutamiento de mensajes**. Pasted markdown (2) Pasted markdown (2)

Y algo importante que encontraste: **no necesariamente es REST vs mensajería de forma absoluta**. Una arquitectura puede combinar comunicación síncrona y asíncrona según la necesidad. Pasted text (4)

Ahora no decidamos todavía

Para tomar la decisión de GameStats nos falta una investigación concreta:

Busca:

```
Kafka vs RabbitMQ vs REST small academic project
```

y también:

```
Kafka RabbitMQ learning curve resource requirements
```

Quiero que veamos **qué tan razonable sería montar cada alternativa para un proyecto pequeño en Go + Scala**, porque ahí está la decisión arquitectónica real.

Después de eso hacemos la matriz de decisión de GameStats y **tú eliges junto conmigo**, no simplemente te digo cuál usar.

Pasted text.txt
Document

RabbitMQ vs. Apache Kafka: Choosing the Right Messaging Backbone for Microservices

Introduction

In microservices architecture, asynchronous communication is critical. While synchronous calls (REST, gRPC) are straightforward, they can tightly couple services and create cascading failures. Messaging systems decouple producers from consumers, making microservices more resilient.

Two dominant technologies in this space: RabbitMQ and Apache Kafka.

Both are widely used at scale but solve different problems. Confusing them—or using the wrong one—can create bottlenecks or architectural issues.

Core Conceptual Difference

✅ RabbitMQ → A message broker that routes messages using queues and exchanges. Optimized for task distribution and low-latency messaging.

✅ Apache Kafka → A distributed event streaming platform for high-throughput, persistent event logs. Optimized for event-driven systems and data streaming.

Think of RabbitMQ as a post office delivering letters, while Kafka is a distributed journal where every event is recorded and replayable.

Architectural Models

RabbitMQ Architecture

✅ Exchange → Receives messages from producers.

✅ Queue → Stores messages until consumed.

✅ Binding → Rules determining which queue(s) an exchange routes messages to.

✅ Consumer → Processes messages from a queue.

👉 Best for command-driven workflows (e.g., "send email," "process payment").

Kafka Architecture

✅ Producer → Writes events to a topic.
✅ Topic → Partitioned, append-only log of events.
✅ Consumer Group → Reads from topics with parallelism across partitions.
✅ Broker → Stores and replicates data.
✅ ZooKeeper/Kraft → Manages cluster metadata.
👉 Best for event-driven systems, streaming pipelines, and analytics.

Message Delivery & Ordering

RabbitMQ

✅ Messages removed from the queue once acknowledged.
✅ Supports at-most-once, at-least-once, and transactional delivery.
✅ Ordering not guaranteed across multiple consumers.

Kafka

✅ Messages retained for a configurable period.
✅ Ordering guaranteed within a partition.
✅ Consumers can replay messages from any offset.
👉 Kafka excels when ordering and replayability matter.

Throughput and Latency

RabbitMQ

✅ Optimized for low-latency messaging (<1 ms typical).
✅ Handles thousands of messages/sec.
✅ Can bottleneck at very high throughput unless tuned.

Kafka

✅ Optimized for high-throughput streaming (millions/sec).
✅ Higher latency (few ms to tens of ms).
👉 RabbitMQ → real-time task execution
👉 Kafka → data pipelines and analytics

Durability and Persistence

RabbitMQ

✅ Messages can be persistent, but queues are memory-first.

✅ Persistence reduces throughput.
✅ Focused on delivery guarantees, not long-term storage.

Kafka

✅ Messages always written to disk (commit log).
✅ Replication ensures fault tolerance.
✅ Designed for long-term storage and event replay.
👉 Kafka is essentially a distributed event database; RabbitMQ is not.

Use Cases in Microservices

RabbitMQ

✅ Task distribution (worker queues)
✅ Request/response over messaging
✅ Event-driven workflows with small scale
✅ IoT device communication
✅ Latency-critical tasks

Kafka

✅ Event sourcing and CQRS
✅ Real-time analytics and monitoring
✅ Streaming data pipelines
✅ Audit logs / Change Data Capture (CDC)
✅ Large-scale pub/sub systems

Operational Complexity

RabbitMQ

✅ Easier to set up and operate
✅ Lightweight, mature ecosystem
✅ Strong .NET support via official client libraries
✅ Limited scalability vs. Kafka

Kafka

✅ Complex deployment and management (clustered, tuning required)
✅ Heavy infrastructure footprint
✅ Best with managed services (Confluent Cloud, Azure Event

Hubs, AWS MSK)

👉 RabbitMQ → developer-friendly

👉 Kafka → enterprise-scale but ops-heavy

Head-to-Head Comparison

Feature	RabbitMQ	Apache Kafka
Primary Model	Message Broker (queues)	Event Streaming Platform (logs)
Latency	Very low (<1ms typical)	Low, but higher than RabbitMQ (ms–10ms)
Throughput	Thousands/sec	Millions/sec
Persistence	Optional, limited	Always persisted, durable
Ordering	Per-queue (limited)	Per-partition (guaranteed)
Replay	Not supported	Fully supported
Scaling	Vertical + limited horizontal	Massive horizontal scaling
Best for	Task distribution, workflows	Event-driven systems, data pipelines

How to Decide

Choose RabbitMQ when:

✅ Fast, lightweight messaging
✅ Task-based workloads
✅ Simple infrastructure
Choose Kafka when:
✅ Stream, store, and replay events
✅ High throughput and scalability
✅ Fundamentally event-driven system
Conclusion
RabbitMQ and Kafka solve different problems:
✅ RabbitMQ → traditional message broker: simple, fast, reliable for task distribution
✅ Kafka → distributed event log: powerful, scalable, for event streaming and replay
In microservices, you may need both: RabbitMQ for real-time workflows, Kafka for event sourcing or analytics pipelines.
👉 Takeaway: Don't ask "Which is better?" — ask "Which fits my system's communication pattern?"
✍️ Written by Saeid Ghaderi
💡 Follow me on dev.to
for more deep dives into microservices, messaging, and cloud-native architecture.

Perfecto. Esta investigación ya nos da una diferencia bastante clara.

Lo importante para GameStats

RabbitMQ

- Está centrado en **mensajes, colas y enrutamiento**.
- Es adecuado para distribuir tareas y manejar flujos con reglas de routing.
- Generalmente tiene menor complejidad operativa.
- También tiene Streams, pero eso ya añade otro modelo. Pasted text

Kafka

- Está diseñado alrededor de **event streams persistentes**.

- Los eventos permanecen y pueden ser **reprocesados/reproducidos**.
- Su modelo de particiones permite mucho paralelismo y throughput.
- A cambio, tiene más conceptos que aprender y operar: particiones, consumer groups, offsets, retención, etc. Pasted text Pasted text

Y aquí aparece algo interesante

GameStats **sí trabaja conceptualmente con eventos**, y uno de los requisitos es simular eventos llegando y procesarlos.

Pero eso **no significa automáticamente que Kafka sea necesario**.

La pregunta que debemos responder es:

> **¿Qué característica de Kafka/RabbitMQ necesitamos realmente para cumplir el proyecto que REST o archivos no nos darían?**

Si la respuesta es "ninguna", meter un broker sería complejidad innecesaria.

Si encontramos una característica que encaja directamente con los requisitos —por ejemplo **procesamiento asíncrono de múltiples partidas, persistencia/reprocesamiento de eventos o desacoplamiento entre Go y Scala**— entonces la mensajería tendría una justificación técnica.

Siguiente investigación

Ahora yo buscaría solamente esto:

```
Go Kafka client Scala Kafka client
```

y

```
Go RabbitMQ client Scala RabbitMQ client
```

Queremos saber **qué tan fácil sería integrar Go → broker → Scala**.

Después de eso ya podemos comparar las **3 alternativas reales de GameStats: REST vs RabbitMQ vs Kafka**, y tomar la decisión.

Investiga eso en internet tu mismo esta vez

Sí. Investigué directamente la integración **Go → RabbitMQ/Kafka → Scala**, usando documentación oficial y librerías actuales.

RabbitMQ

La integración es bastante directa:

Go

- Existe `amqp091-go`, mantenido por el propio equipo de RabbitMQ.
- Permite publicar y consumir mensajes desde Go. GitHub +1

Scala

- Scala puede utilizar directamente el cliente oficial de RabbitMQ para Java porque corre sobre la JVM.
- RabbitMQ incluso lista explícitamente Scala entre los lenguajes que pueden usar su cliente Java. RabbitMQ +1

Arquitectura:

```
Simulador
   ↓
  Go
   ↓
RabbitMQ
   ↓
 Scala
```

Kafka

También es viable:

Go

- IBM mantiene **Sarama**, una librería Go para Kafka con APIs para producir y consumir mensajes. Go Documentation +1

Scala

- Al ser JVM, Scala puede utilizar clientes Java de Kafka.

Arquitectura:

```
Plain text

Simulador
   ↓
  Go
   ↓
 Kafka Topic
   ↓
 Scala
```

Lo que me parece más importante para nuestro proyecto

La integración técnica **no elimina ninguna de las dos opciones**.

Pero sí aparece una diferencia:

- **RabbitMQ:** Go publica un evento → Scala lo consume.
- **Kafka:** Go publica eventos → quedan en un log/topic → Scala puede consumirlos y posteriormente volver a procesarlos.

Para GameStats, esa segunda característica podría ser interesante porque estamos trabajando precisamente con **eventos que después serán analizados**.

Pero todavía **no elegiría Kafka solo por eso**. Tenemos que determinar si realmente necesitamos *replay/persistencia de eventos* o si simplemente necesitamos transportar los eventos de Go a Scala.

Siguiente paso

Ahora sí tenemos información suficiente para hacer la comparación final:

REST vs RabbitMQ vs Kafka específicamente para GameStats, usando los requisitos del profesor como criterios.

Ahí podremos justificar la decisión técnicamente en el documento, en lugar de elegir una tecnología porque "suena más profesional".

Exacto. Ya no necesitamos seguir acumulando artículos sobre RabbitMQ y Kafka.

El siguiente paso lógico es construir la **matriz de decisión de GameStats** con:

1. Cumplimiento del requisito Go ↔ Scala.
2. Procesamiento de eventos simulados.
3. Concurrencia de múltiples partidas.
4. Persistencia/reprocesamiento.
5. Complejidad de implementación.
6. Facilidad para demostrarlo al profesor.
7. Docker/CI/CD.
8. Justificación académica.

Con eso podemos llegar a una decisión concreta entre **REST, RabbitMQ y Kafka** y documentar por qué.

Perfecto cual seria tu veredicto final pros y contras

Mi veredicto para **GameStats**, considerando el alcance académico y los requisitos del proyecto:

1. REST

Pros

- Muy sencillo de implementar.
- Go y Scala pueden comunicarse fácilmente mediante HTTP + JSON.
- Fácil de probar con herramientas como Postman/curl.
- Menos infraestructura.
- Facilita Docker y CI/CD.
- Cumple el requisito de componentes Go y Scala independientes.

Contras

- Comunicación principalmente síncrona.
- Go tiene que comunicarse directamente con Scala.
- Menos natural si queremos representar un flujo continuo de eventos.

- No proporciona por sí mismo cola, persistencia de mensajes o replay.

---

2. RabbitMQ

Pros

- Comunicación asíncrona.
- Go y Scala quedan desacoplados.
- Encaja muy bien con el modelo productor → consumidor.
- Permite manejar múltiples eventos y consumidores.
- Tiene routing mediante exchanges/queues.
- Relativamente sencillo de entender y operar frente a Kafka. Pasted text

Contras

- Tenemos que agregar RabbitMQ como infraestructura.
- Hay que manejar conceptos adicionales: exchanges, queues, bindings, acknowledgements, etc.
- Introduce problemas adicionales como duplicados y ordenamiento.
- Para nuestro proyecto puede ser más infraestructura de la estrictamente necesaria.

---

3. Kafka

Pros

- Está especialmente orientado a **eventos y streaming**.
- Los eventos pueden permanecer almacenados y ser reprocesados.
- El modelo de topics/partitions encaja muy bien con grandes flujos de eventos.
- Go → Kafka → Scala representa muy bien una arquitectura event-driven.
- Permite que diferentes consumidores analicen los mismos eventos independientemente.

Pasted text

Contras

- Mayor complejidad conceptual y operacional.
- Tenemos que entender topics, partitions, consumer groups, offsets, retención, etc.

Pasted text

- Para el volumen de un proyecto académico, muchas de sus capacidades pueden quedar sin utilizar.
- Añade bastante infraestructura al proyecto.

Mi decisión

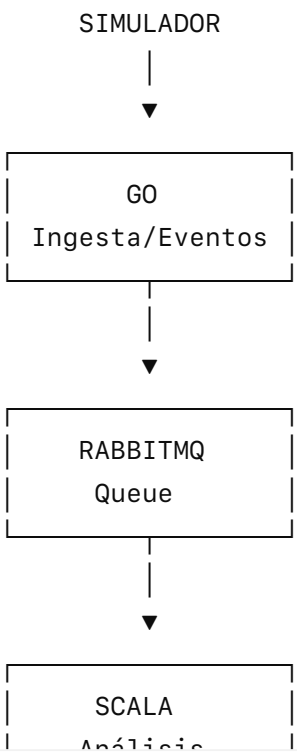
Para **GameStats**, yo escogería **RabbitMQ**.

No porque sea "mejor" en general, sino porque veo el mejor equilibrio entre:

eventos + comunicación asíncrona + Go/Scala desacoplados + concurrencia + complejidad razonable.

La arquitectura quedaría:

Plain text



REST sería la opción más simple. Kafka sería la opción más orientada a streaming a gran

escala. RabbitMQ queda en medio, y para este proyecto académico esa característica pesa bastante.

Además, podemos justificarlo ante el profesor técnicamente, no simplemente diciendo *"porque RabbitMQ es popular"*.

Podriamos usar REST para no aumentar la complejidad

Sí. **De hecho, para GameStats considero perfectamente defendible usar REST**, y simplificaría bastante el proyecto.

El requisito del profesor no dice que debamos usar un broker; permite **REST, archivos estructurados o mensajería** para comunicar Go y Scala. Proyecto1 - GameStats_Enunciado...

Cómo quedaría

Plain text

```
Simulador de eventos
        ↓
       Go
  ingesta + validación
        ↓
     HTTP/REST
        ↓
      Scala
 análisis + estadísticas
        ↓
   resultados JSON
```

¿Qué perdemos frente a RabbitMQ?

Principalmente:

- Cola de mensajes.
- Comunicación naturalmente asíncrona.
- Persistencia/replay proporcionados por el middleware.
- Desacoplamiento adicional entre productor y consumidor.

Pero **ninguno de esos elementos aparece como requisito obligatorio del proyecto**.

¿Qué ganamos?

- Menos infraestructura.
- Menos conceptos que aprender simultáneamente.
- Menos código.
- Docker más sencillo.
- Debugging más sencillo.
- Demo más sencilla.
- Podemos concentrarnos en lo que **sí evalúa el proyecto**: Go, concurrencia, eventos, Scala funcional, estadísticas, patrones y comunicación entre componentes.

Proyecto1 - GameStats_Enunciado...

Y REST no impide que Go procese múltiples partidas concurrentemente. La concurrencia puede estar **dentro de Go**, usando goroutines/channels, que sí es parte explícita del requisito.

Proyecto1 - GameStats_Enunciado...

Mi cambio de decisión

Si nuestra prioridad es **hacer un proyecto académico sólido sin sobrearquitecturarlo**, entonces:

> **REST + JSON sería mi elección para GameStats.**

RabbitMQ sería interesante técnicamente, pero tendríamos que demostrar que esa complejidad adicional aporta algo necesario. Con los requisitos actuales, no parece indispensable.

La siguiente decisión sería definir **exactamente qué recibe el endpoint REST de Scala y qué devuelve**, antes de empezar a programar.

Perfecto esa se queda como la esctructura

Perfecto. Queda definida la arquitectura de comunicación de **GameStats**:

```
SIMULADOR DE EVENTOS
        ↓
       GO
  Ingesta + validación
  + concurrencia
        ↓
     HTTP / REST
        ↓
      SCALA
  Análisis funcional
  + estadísticas
  + patrones
        ↓
   RESULTADOS JSON
```

La decisión queda: **REST + JSON**, sin RabbitMQ ni Kafka.

Ahora podemos pasar a la siguiente parte de arquitectura: **definir el contrato de los eventos y el contrato REST entre Go y Scala**.

Si continua

Sí. Ahora vamos con el **contrato de eventos**, pero todavía sin programar.

1. Evento base

El brief establece que los eventos deben tener información común como:

```
eventId
timestamp
gameId / gameVersion
```

```
matchId
playerId
type
action
data
```

Proyecto1 - GameStats_Enunciado...

La idea sería separar:

- **Metadatos comunes** → identifican el evento.
- `data` → información específica de cada tipo de evento.

Ejemplo conceptual:

JSON

```json
{
  "eventId": "evt-001",
  "timestamp": "2026-09-30T18:20:15Z",
  "gameId": "game-01",
  "gameVersion": "1.0",
  "matchId": "match-001",
  "playerId": "player-07",
  "type": "ACTION",
  "action": "SCORE",
  "data": {
    "points": 2
  }
}
```

2. ¿Por qué esta estructura?

Porque queremos que Go pueda recibir eventos de **diferentes juegos sin cambiar completamente su arquitectura**.

Por ejemplo:

```
Juego A → SCORE
Juego B → GOAL
Juego C → ELIMINATION
```

pero todos conservan los identificadores comunes.

El brief justamente exige que el diseño sea suficientemente genérico para manejar distintos juegos/eventos. Proyecto1 - GameStats_Enunciado...

3. Lo que todavía NO vamos a decidir

Todavía nos falta definir:

- qué tipos de juegos vamos a simular;
- qué eventos tendrá cada uno;
- qué estadísticas calcularemos;
- qué secuencias/patrones analizaremos.

Eso es importante porque **el `data` de los eventos dependerá de esas decisiones**.

Así que el siguiente paso debería ser:

> **Investigar y seleccionar los 2 tipos de juegos que utilizaremos como referencia para diseñar nuestros eventos.**

No implementamos nada hasta tener eso definido.